

청강대 게임 프로그래밍 지원 · PORTFOLIO

박재영

플레이하며 생긴 궁금증을
직접 구현하며 배웁니다.

리듬 게임의 시간 계산부터 액션 게임의 전투와 재도전까지 구현했습니다. 화면과 핵심 코드를 함께 보며 자료구조, 상태 전환, 이벤트 연결을 어떻게 사용했는지 설명합니다.

C++ · Unreal Engine

C# · Unity

Python · Pygame

미림마이스터고 뉴미디어소프트웨어과
CodeUs 동아리 부장 · 6인 팀 메인 프로그래머 경험



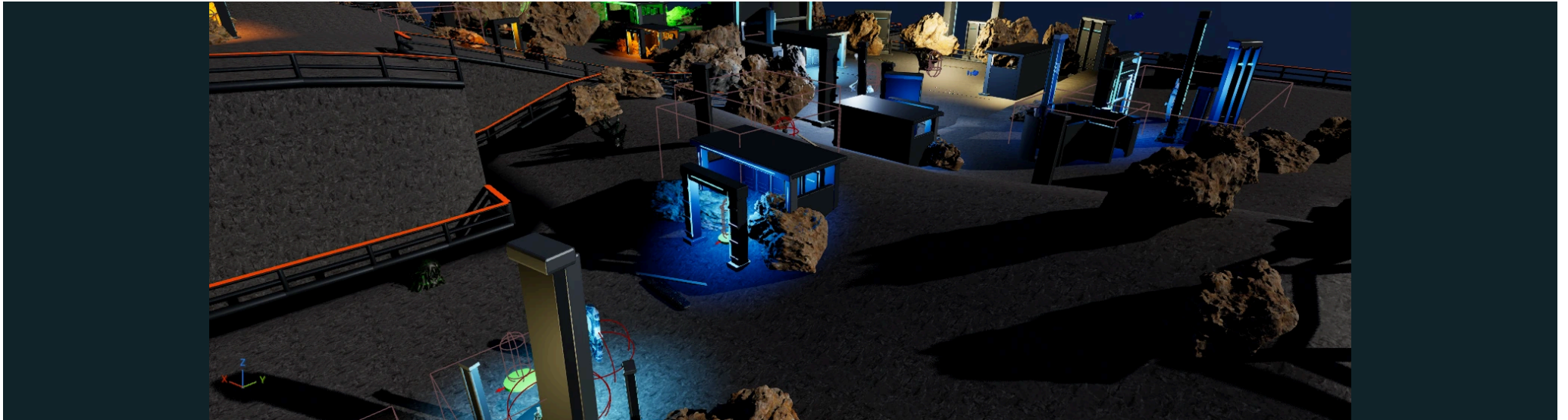
Exception · 플레이어와 HUD



Error Dungeon · 런타임 월드 / HUD 제외

회피와 패링으로 기회를 만드는 액션 RPG

손상된 Runtime을 탐험하고, 보스의 빈틈을 노려 Groggy와 처형으로 이어지는 전투를 만들고 있습니다.



필드 구조물과 탐험 동선 · 에디터 배치 화면

기간 2026.06 ~ 진행 중 형태 개인 프로젝트 담당 기획 · 게임플레이 프로그래밍

GAME

보스전에 집중한 플레이

탐험, 전투, 사망 후 체크포인트 복귀로 이어지는 액션 RPG입니다.

MY WORK

전투 · 보스 · UI · 저장

공격·회피·패링, 보스 상태, HUD 갱신과 저장·로드를 구현했습니다.

CHALLENGE

서로 영향을 주는 상태

한 번의 공격에서의 중복 피해, 두 보스의 공격 시점과 간격, 처형 중 상태 전환을 다뤘습니다.

RESULT

전투와 복귀 흐름 연결

공격별 적중 이력과 보스 조율자, 이벤트 기반 HUD를 연결했습니다. 현재 개발을 이어가고 있습니다.

판정 횟수와 피해 횟수를 분리했습니다.

범위 검사로 대상을 찾고, 공격 단위로 기록해 같은 대상에게 피해가 겹치지 않게 했습니다.



플레이어와 HUD · 플레이 화면



듀얼 보스의 외형과 배치 · 에디터 화면

한 번의 공격, 한 번의 피해

문제 같은 대상이 여러 충돌 결과에 포함될 수 있습니다.

구현 Sphere Sweep으로 후보를 찾고 TSet으로 공격별 적중 대상을 관리합니다. 피해가 적용된 대상만 기록합니다.

결과 같은 공격 안에서는 중복 대상을 건너뛰고, 다음 공격에서 이력을 초기화합니다.

두 보스의 공격 시점과 간격 조율

문제 각 보스가 독립적으로 공격하면 공격 시점이 겹칠 수 있습니다.

구현 공용 조율자에서 현재 공격자와 다음 공격 가능 시각을 검사합니다.

결과 동시 공격 허용 여부와 공격 간격을 한곳에서 결정합니다.

처형 중 상태 전환 · Groggy 회복 시 처형 상태를 확인하고, 처형 중이면 회복 타이머를 다시 예약합니다.

피해가 적용된 대상만 공격 이력에 기록합니다.

범위에 들어온 대상, 이미 맞은 대상, 실제로 피해를 받은 대상을 구분했습니다.

PlayerFight.cpp

C++

```
573 TArray<FHitResult> Hits;
574 const FCollisionShape Shape = FCollisionShape::MakeSphere(AttackTraceRadius);
575 const bool bHit = World->SweepMultiByObjectType(Hits, TraceStart, TraceEnd,
    FQuat::Identity, ObjectParams, Shape, QueryParams);
```

... L576-588

```
589 for (const FHitResult& Hit : Hits)
590 {
591     AActor* HitActor = Hit.GetActor();
592     const TWeakObjectPtr<AActor> HitActorPtr(HitActor);
593     if (!HitActor || DamagedActorsThisAttack.Contains(HitActorPtr))
594     {
595         continue;
596     }
```

... L597-612

```
613 if (!bDamageApplied)
614 {
615     continue;
616 }
617
618 DamagedActorsThisAttack.Add(HitActorPtr);
```

전방 검사 → 중복 확인 → 피해 성공 시 기록. 생략 구간에는 피해 적용 분기가 있습니다.

[L573-575 원문 ↗](#) · [L589-596 원문 ↗](#) · [L613-618 원문 ↗](#)



Exception · 플레이어와 HUD

01 / 대상 수집

한 번의 검사에서 여러 대상을 찾기

Sphere Sweep으로 전방 범위의 충돌 결과를 수집합니다. 이후 각 결과를 순회하며 액터 단위로 판단합니다.

02 / 자료구조 선택

같은 공격의 중복 피해 차단

TSet의 Contains로 적중 이력을 검사합니다. 같은 액터가 여러 충돌 결과에 포함되어도 이미 기록된 대상은 건너뜁니다.

03 / 기록 시점

성공한 피해만 다음 판정에서 제외

bDamageApplied가 참일 때만 이력에 추가합니다. 피해가 적용되지 않은 대상까지 제외하지 않도록 기록 시점을 뒤로 두었습니다.

배운 점 · 다음 과제

충돌 검출과 게임 규칙의 성공 여부는 다릅니다. 공격을 시작할 때 이력을 비우는 위치까지 함께 확인해야 합니다.

처형이 끝나기 전에는 Groggy 회복을 미룹니다.

상태를 바꾸기 전에 우선순위를 확인하고, 진행 중인 처형을 유지했습니다.

BossFight.cpp

C++

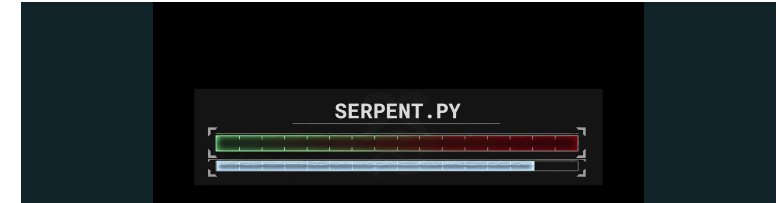
```

309 void ABRBossBase::RecoverFromGroggy()
310 {
311     if (bIsDead || bIsPhaseTransitioning || !bIsGroggy)
312     {
313         return;
314     }
315
316     if (bIsBeingExecuted)
317     {
318         GetWorldTimerManager().SetTimer(GroggyTimerHandle, this,
&ABRBossBase::RecoverFromGroggy, 0.1f, false);
319         return;
320     }
321
322     bIsGroggy = false;
323     NotifyBossAnimationStage(EBRBossAnimationStage::Idle);
324
325     if (StatComponent)
326     {
327         StatComponent->ResetGroggy();
328     }
329
330     OnBossRecoveredFromGroggy.Broadcast();
331     OnBossRecoveredFromGroggyInternal();

```

RecoverFromGroggy의 상태 검사와 복구 처리. 함수 끝 닫는 중괄호는 다음 줄에 있습니다.

[L309-331 원문 ↗](#)



Exception · 보스 HP와 Groggy 표시

01 / 조기 반환

회복할 수 없는 상태부터 제외

사망했거나 페이즈 전환 중이거나 이미 Groggy 상태가 아니면 함수를 종료합니다. 아래의 회복 로직에 진입할 조건을 먼저 좁힙니다.

02 / 재시도

처형 중이면 0.1초 뒤 다시 확인

처형 상태에서는 타이머를 다시 등록하고 반환합니다. 회복 시간이 도달해도 현재 동작의 상태를 즉시 덮어쓰지 않습니다.

03 / 상태와 알림

수치를 복구한 뒤 변경을 전달

Groggy 플래그, 애니메이션 단계, 수치를 정리한 뒤 회복 이벤트를 보냅니다. 상태 변경과 외부 알림의 순서를 드러냈습니다.

배운 점 · 다음 과제

조건문의 순서가 플레이 흐름을 결정합니다. 처형·사망·페이즈 전환이 연속으로 발생하는 경우도 확인할 필요가 있습니다.

상태가 바뀌면 화면과 저장 데이터도 따라갑니다.

C++에서 수치와 상태를 관리하고, Blueprint에서 UI 배치와 연출을 연결했습니다.



아이템 분류와 상세 정보 · 플레이 중 인벤토리



일시정지 메뉴와 레벨업 패널 · 플레이 화면

플레이어가 바뀌어도 HUD 갱신

문제 Pawn을 교체하면 이전 캐릭터를 바라보던 UI 연결을 정리해야 합니다.

구현 기존 Delegate를 해제하고 새 Pawn의 상태 변화에 바인딩한 뒤 현재 수치를 즉시 갱신합니다.

결과 캐릭터 상태를 HUD에 전달하는 연결과 화면 표시의 책임을 나눴습니다.

새 레벨의 플레이어에 저장 상태 적용

문제 레벨 이동 뒤에는 위치뿐 아니라 능력치와 인벤토리도 복구해야 합니다.

구현 체크포인트를 우선해 복귀 위치를 정하고, 저장된 상태를 새 플레이어에 적용합니다.

결과 위치·능력치·인벤토리의 복구를 하나의 로드 흐름으로 묶었습니다.

HUD 연결은 2026.09.11 작업본 기준입니다. 저장-로드는 아래 구현에서 확인할 수 있습니다.

플레이어의 상태 변화에 HUD를 연결합니다.

화면이 직접 전투 규칙을 판단하지 않고, 상태가 바뀌었다는 이벤트를 받아 표시하게 했습니다.

ExceptionPlayerControllerHUD.cpp

C++

```

112 void AExceptionPlayerController::BindPlayerHUDToPawn()
113 {
114     if (!IsLocalPlayerController() || IsInTitleLevel())
115     {
116         return;
117     }
118
119     AExceptionCharacter* ExceptionCharacter = Cast<AExceptionCharacter>
(GetPawn());
120     if (!ExceptionCharacter || BoundHUDCharacter == ExceptionCharacter)
121     {
122         return;
123     }
124
125     BoundHUDCharacter = ExceptionCharacter;
126     BoundHUDCharacter->OnHPChanged.AddUniqueDynamic(this,
&AExceptionPlayerController::HandlePlayerHPChanged);
127     BoundHUDCharacter->OnStaminaChanged.AddUniqueDynamic(this,
&AExceptionPlayerController::HandlePlayerStaminaChanged);
128     BoundHUDCharacter->OnCombatStateChanged.AddUniqueDynamic(this,
&AExceptionPlayerController::HandlePlayerCombatStateChanged);
129 }

```

2026.09.11 로컬 작업본 · 공개 커밋과 다른 HUD 연결 코드입니다.



Exception · 플레이어 HUD

01 / 연결 대상

로컬 플레이어의 유효한 Pawn만 연결

로컬 컨트롤러인지, 타이틀 레벨인지 먼저 확인합니다. Pawn을 캐릭터로 변환한 뒤 유효한지 검사합니다.

02 / 중복 연결

같은 캐릭터에는 다시 바인딩하지 않기

현재 연결된 캐릭터와 같으면 반환합니다. AddUniqueDynamic으로 같은 이벤트 핸들러가 중복 등록되지 않게 합니다.

03 / 역할 분리

HP · 스테미나 · 전투 상태를 각각 전달

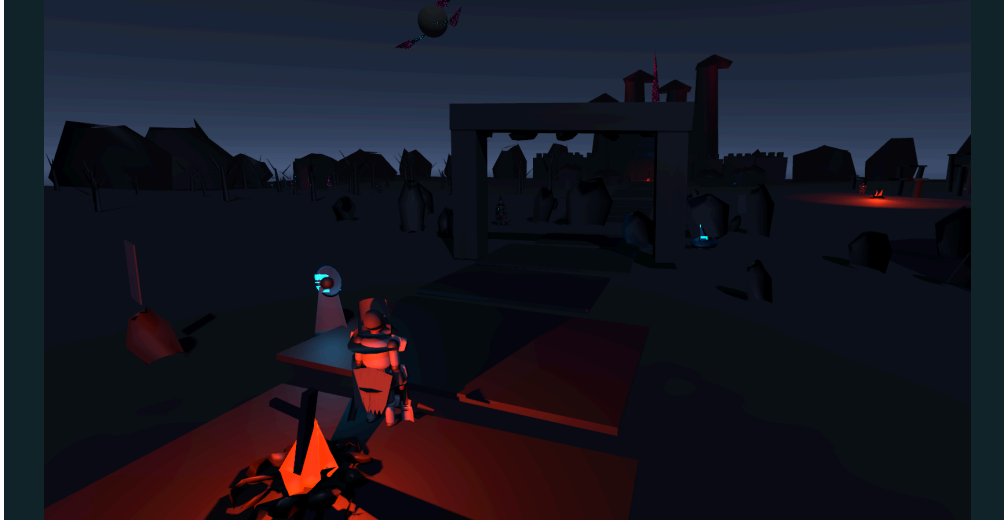
각 이벤트에 대응하는 핸들러를 연결합니다. 전투 데이터의 변경과 HUD가 수치를 그리는 책임을 나누었습니다.

배운 점 · 다음 과제

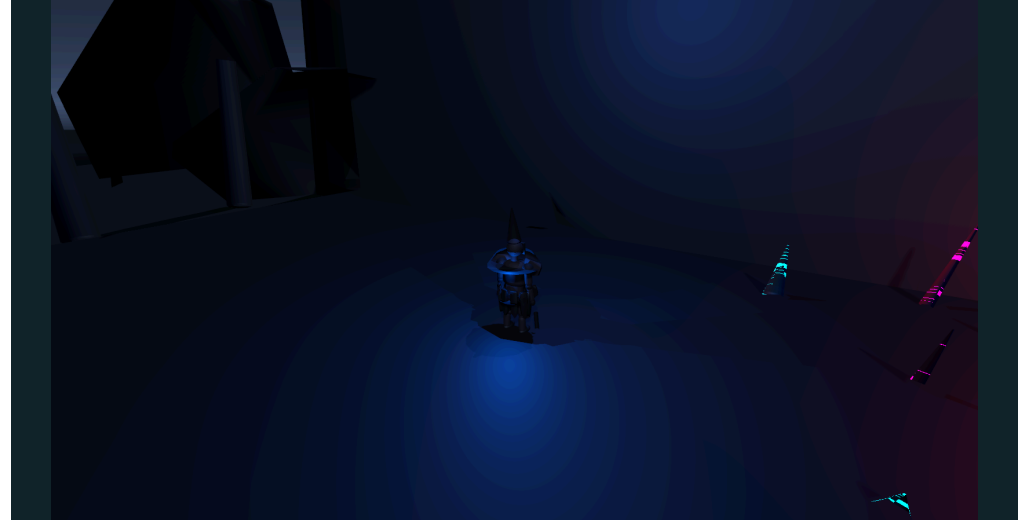
이벤트 등록뿐 아니라 해제 시점도 중요합니다. Pawn이 바뀌는 흐름에서는 이전 연결 정리와 최초 수치 갱신까지 함께 다뤄야 합니다.

공격을 이어가고, 실패해도 다시 도전하는 게임

폐성을 탐험하며 보스전에 도전하는 개인 프로젝트입니다. 전투 입력과 사망 후 복귀 흐름을 구현했습니다.



시작 지역과 플레이어 · 런타임 월드 / HUD 제외



보스와 전투 구역 · 런타임 월드 / HUD 제외

기준 2026.09 작업 기록 담당 게임플레이 · WebGL 배포 환경 Unity · C# · FastAPI

입력 버퍼로 3타 콤보 연결

문제 공격이 끝나기 전에 누른 입력도 다음 타격에 반영해야 합니다.

구현 입력을 버퍼에 저장하고 회복 구간 뒤 스테미너를 확인합니다.

결과 조건이 맞으면 다음 타격으로, 아니면 이동 상태로 돌아갑니다.

잃을 상태와 유지할 진행을 구분

문제 사망 뒤 재화, 적, 처치한 보스의 상태를 다르게 처리해야 합니다.

구현 재화를 사망 위치에 남기고 체크포인트에서 플레이어를 복구합니다.

결과 일반 적은 초기화하되 이미 처치한 보스의 진행은 유지합니다.

저장한 입력과 비용을 확인해 다음 타격을 결정합니다.

현재 공격의 회복 구간 뒤, 다음 콤보로 갈지 이동 상태로 돌아갈지 선택하는 부분입니다.

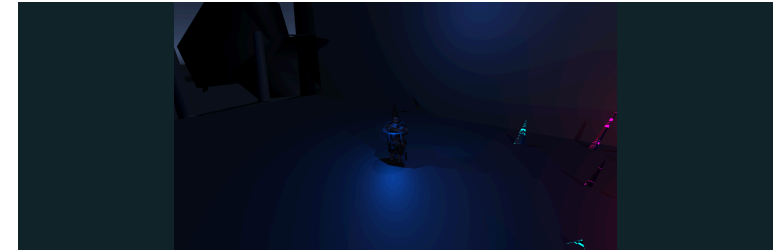
PlayerCombat.cs

C#

```
162 var nextStep = (step + 1) % 3;
163 nextComboStep = nextStep;
164 comboExpiresAt = Time.time + (step == 2 ? 0.36f : 0.58f);
165 if (!lightAttackBuffered)
166 {
167     activeComboStep = -1;
168     attackRoutine = null;
169     state.ReleaseToLocomotion();
170     yield break;
171 }
172
173 if (!vitals.TrySpendStamina(LightStamina(nextStep)))
174 {
175     activeComboStep = -1;
176     attackRoutine = null;
177     state.ReleaseToLocomotion();
178     yield break;
179 }
180 step = nextStep;
```

LightComboRoutine의 다음 타격 결정 구간. 앞쪽 공격 준비·적중 처리와 함수 끝부분은 생략했습니다.

[L162-180 원문 ↗](#)



Error Dungeon · 보스 구역 / HUD 제외

01 / 단계 계산

다음 타격과 연결 가능 시간 갱신

(step + 1) % 3으로 콤보 단계를 순환합니다. 마지막 타격인지에 따라 다음 입력을 이어갈 수 있는 시간을 다르게 설정합니다.

02 / 입력 확인

버퍼가 비어 있으면 이동으로 복귀

공격 도중 들어온 입력은 앞선 처리에서 저장합니다. 버퍼가 없으면 콤보 상태와 코루틴 참조를 정리한 뒤 종료합니다.

03 / 자원 검사

다음 타격의 스테미나를 지불할 수 있는가

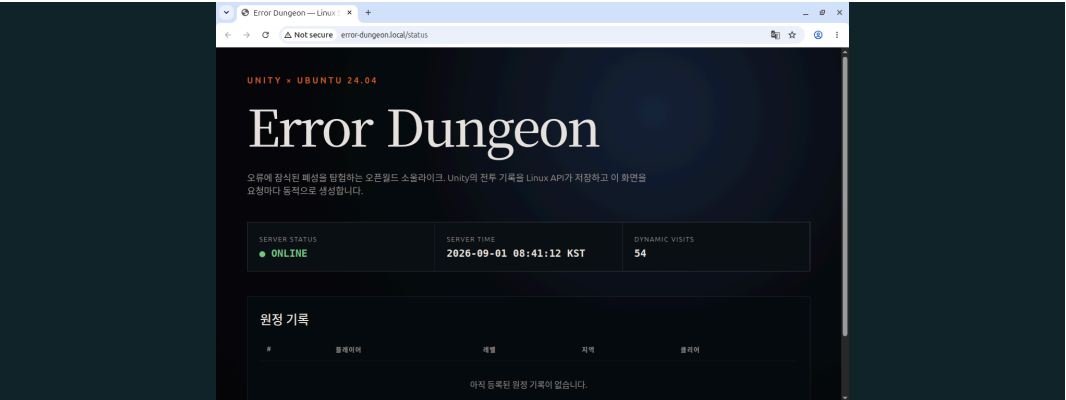
TrySpendStamina가 성공할 때만 다음 단계로 넘어갑니다. 실패 경로에서도 동일하게 상태를 정리합니다.

배운 점 · 다음 과제

입력을 누른 시점과 행동이 시작되는 시점은 다를 수 있습니다. 입력 버퍼와 상태 정리가 함께 있어야 콤보 흐름을 설명할 수 있습니다.

배포했는데 예전 화면이 보이는 문제를 추적했습니다.

WebGL 클라이언트와 API를 로컬 Linux VM에 배포하고, 파일과 응답을 확인하며 문제를 좁혔습니다.



로컬 Linux VM의 상태 페이지 · 2026.09 배포 기록

새 빌드와 실행 화면 불일치

문제 새 WebGL을 올렸는데 이전 모습이 보였습니다.

구현 로컬·배포 wasm의 SHA-256과 응답 헤더를 비교했습니다. /Build/에 no-cache를 적용하고 클린 빌드를 rsync로 재배포했습니다.

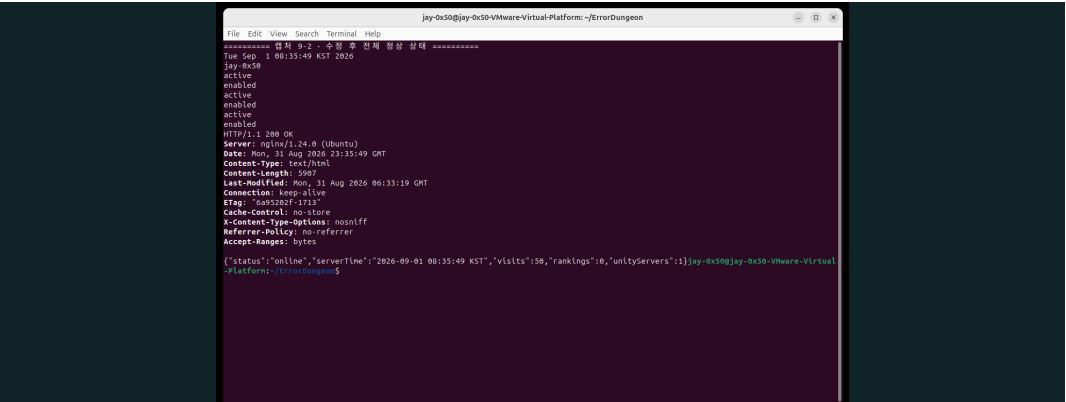
결과 배포 기록에서 로컬과 서버의 파일 해시가 일치하는 것을 확인했습니다.

API 재시작 중 일시적 502

문제 서비스를 재시작하는 순간 API 점검이 실패했습니다.

구현 systemctl과 journalctl로 상태를 확인하고, curl 점검에 재시도를 추가했습니다.

결과 배포 당시 웹·API의 HTTP 200 응답과 3개 서비스의 active 상태를 확인했습니다.



배포 후 서비스 상태와 HTTP 200 응답 확인

VMware NAT-hosts 기반 로컬 환경. AI 도구의 코드·분석 보조를 활용하고, VM 배포 실행과 응답·로그 검증을 진행했습니다.

재진입할 때 먼저 재생되던 음악의 시작 조건을 고쳤습니다.

곡 선택부터 플레이, 콤보.점수 계산과 결과 확인까지 한 프로그램으로 만들었습니다.



PyMax 곡 선택 화면



4키 노트 · 판정선 · 콤보 · 피버 게이지

기간 2025.06 형태 개인 프로젝트 담당 프로그래밍

GAME / MY WORK

4키 리듬 게임

채보 로드, 노트 이동, 입력 판정과 콤보.점수.
피버 게이지를 구현했습니다.

PROBLEM

다시 들어가면 조기 재생

뒤로 갔다가 재진입하면 이미지·음원은 이미 준비됐는데 로딩 대기는 그대로 남아, 노래와 영상이 먼저 재생됐습니다.

SOLUTION

시간 대신 준비 상태 확인

정해진 로딩 대기 시간에 맞추던 시작 조건을 리소스 로딩 완료 기준으로 변경했습니다.

RESULT

로딩 완료 뒤 재생 시작

재진입 시 미디어가 먼저 재생되는 문제를 수정했습니다. 화면의 대기 시간과 실제 준비 상태를 구분하는 중요성을 배웠습니다.

같은 시간 데이터로 노트 위치와 입력 결과를 계산합니다.

채보에 저장한 목표 시각을 화면 좌표와 판정 기준으로 사용했습니다.

PyMax.py

Python

```
865 for tile_data in t1:
866     tile_data[0] = ((h / 24) * 17) + (Time + sst - tile_data[1]) * 350 * speed *
      (h / 900)
867     if tile_data[0] + note.get_height() > 0:
868         screen.blit(note, (w / 2 - w / 10, tile_data[0] - note.get_height() / 2))
```

노트 도착 시각과 경과 시간의 차이로 Y좌표를 계산하고 화면에 그림니다.

[L865-868 원문 ↗](#)

PyMax.py

Python

```
167 if abs(Time - rate_data[n - 1]) < 240 / bpm / 32 and abs(Time - rate_data[n -
      1]) >= 0:
168     combo_effect = 0.2
169     combo_time = Time + 1
170     combo_effect2 = 1.3
171     rate = "PERFECT"
172     combo += 1 * fever
173     effect_key[n - 1] = 0
174     effect_key_size[n - 1] = 13
175     score += (100000 / note_count) * 1
176     fever_gauge += 1
```

rating 함수의 PERFECT 구간. 다른 판정 등급의 분기는 생략했습니다.

[L167-176 원문 ↗](#)



PyMax · 노트와 판정선, 콤보

01 / 화면 좌표

남은 시간에 속도와 화면 비율 반영

노트 시각과 현재 시간의 차이에 속도와 해상도 비율을 곱합니다. 이동량을 계속 누적하는 대신 시간값에서 위치를 계산합니다.

02 / 입력 판정

노트와 입력 시각의 차이를 비교

`abs(Time - rate_data[n - 1])`로 시간 오차를 구합니다. PERFECT 범위는 $240 / \text{bpm} / 32$ 로 정해집니다.

03 / 피드백

판정에 맞춰 점수와 화면 상태 갱신

판정 문자열, 콤보, 키 이펙트, 점수와 피버 게이지를 함께 갱신해 입력 결과를 플레이어에게 전달합니다.

배운 점 · 다음 과제

BPM에 따라 판정의 시간 폭이 달라집니다. 다음에는 판정 폭과 템포를 분리할지 검토하고, 항상 참인 `abs(...) >= 0` 조건도 정리하고 싶습니다.

만들며 배운 것, 더 깊이 배우고 싶은 것

직접 구현한 동작을 설명하고, 플레이 테스트로 설계를 검증하는 개발자로 성장하고 싶습니다.

EXCEPTION

기능 사이의 연결

전투·보스·HUD·저장을 연결하며 데이터의 주인과 갱신 시점을 구분했습니다. 상태 전환과 예외 상황을 더 체계적으로 검증하고 싶습니다.

PYMAX

시간과 조작감

노트 위치와 입력 시점을 다루며 작은 계산이 게임의 반응으로 이어지는 과정을 배웠습니다. 오디오 동기화와 판정 범위를 더 탐구하고 싶습니다.

ERROR DUNGEON

동작에서 배포까지

전투 입력과 재도전을 구현하고, WebGL 배포 과정에서 파일·HTTP 응답·로그를 확인했습니다. 실행 환경을 이해하는 공부도 이어가겠습니다.

SPACEOUT

엔진 아래의 원리

학원 예제를 실습하며 입력·충돌·출력 순서를 따라갔습니다. 자료구조와 그래픽스를 공부해 동작의 이유를 설명하고 싶습니다.

청강대에서 이어가고 싶은 공부

기초와 구현 연결

C++·자료구조·선형대수를 공부하고 충돌 판정과 렌더링에 적용하겠습니다.

팀으로 작품 완성

기획·아트와 프로토타입을 함께 보며 의도를 맞추고 기능을 연결하겠습니다.

플레이 테스트와 개선

피드백을 기록하고 수정 전후를 비교하며 한 작품의 완성도를 높이겠습니다.

게임을 만드는 원리를 넓혀 왔습니다.

학습 프로젝트는 실습한 범위를, 다른 프로젝트는 직접 맡은 기능을 중심으로 정리했습니다.



SpaceOut · 학원 예제 기반 Win32 슈팅 실습

C++ · Win32 API · GDI / 2025.12

SpaceOut

입력에 따른 이동·탄환 생성, 스프라이트 갱신, AABB 충돌과 GDI 화면 출력을 실습했습니다.

두 사각형이 가로·세로 축에서 모두 겹치는지 확인해 충돌을 판정하고, 오프스크린에 합성한 화면을 출력하는 과정을 익혔습니다.

MopCode · C++17

토큰화, 표현식과 제어문 실행을 구현한 실험적 스크립팅 언어.

[저장소 ↗](#)

NullEngine · C++17 / OpenGL

윈도우 수명주기, 셰이더와 버퍼 추상화를 구현하며 엔진 구조를 학습.

[저장소 ↗](#)

NEXT STEP

만들고, 플레이하고, 고치는 과정을 이어가겠습니다.

C++ · Unreal Engine · C# · Unity

jaerin.park.08@gmail.com

[GitHub ↗](#)